

Part 3: Logging & Monitoring — Notes

WHAT WE DID AND WHY

The Problem

Before Part 3, the entire backend used `print()` for logging:

python

```
print(f"✅ Added document to PostgreSQL: {doc_id[:8]}...")
print(f"❌ Error adding document: {e}")
```

Problems with `print()`:

- No timestamps — you don't know WHEN something happened
- No log levels — you can't tell if it's an error or just info
- No file logging — if the server crashes, all prints are lost forever
- No module info — you don't know WHICH file generated the log
- Can't filter — you can't say "show me only errors"
- Looks unprofessional in production

The Solution

We replaced all `print()` with Python's logging module:

python

```
logger.info(f"Added document to PostgreSQL: {doc_id[:8]}...")
logger.error(f"Error adding document: {e}")
```

Now logs look like:

2026-02-16 19:17:47 | INFO | metadata_store | Added document to PostgreSQL: abc123...

2026-02-16 19:18:06 | ERROR | metadata_store | Error adding document: timeout

2026-02-16 19:18:06 | WARNING | cache | Cache miss for query

2026-02-16 19:18:07 | INFO | api | POST /api/query → 200 (234.56ms)

FILES CREATED / MODIFIED

NEW FILES:

— core/logger.py	→ Central logging system
— app/api/health.py	→ Health check endpoints
— app/middleware.py	→ Auto-logs every API request

MODIFIED FILES:

— app/main.py	→ Added middleware, health routes, startup logs
— core/metadata_store.py	→ <code>print()</code> → logger
— core/cache.py	→ <code>print()</code> → logger
— core/retriever.py	→ <code>print()</code> → logger
— core/embeddings.py	→ <code>print()</code> → logger
— core/vector_store.py	→ <code>print()</code> → logger
— llm/gemini_client.py	→ <code>print()</code> → logger
— Dockerfile	→ Added logs/ directory

HOW EACH FILE WORKS

1. core/logger.py — The Central Logger

This is the brain of the logging system. It creates loggers that:

- Print to terminal (INFO level and above)
- Save to file in logs/ folder (DEBUG level and above — captures everything)
- Add timestamp, level, and module name to every log

Usage in any file:

python

```
from core.logger import get_logger
logger = get_logger(__name__)

logger.debug("Detailed info for debugging") # Only in log file
logger.info("Normal operation info")       # Terminal + file
```

```
logger.warning("Something might be wrong")    # Terminal + file
logger.error("Something definitely broke")    # Terminal + file
```

Log levels (from lowest to highest):

DEBUG → Detailed debugging info (only saved to file)
INFO → Normal operations (document added, query processed)
WARNING → Something unexpected but not broken (cache miss)
ERROR → Something failed (database error, API error)

2. `app/api/health.py` — Health Check Endpoints

Two endpoints that check if the system is working:

GET /health

→ Returns: {"status": "healthy"}
→ Used by: Quick "is it alive?" checks

GET /health/detailed

→ Returns: Status of PostgreSQL, Redis with response times
→ Used by: Docker health checks, monitoring dashboards
→ Example response:

```
{
  "status": "healthy",
  "services": {
    "postgresql": {"status": "connected", "response_time_ms": 2.5},
    "redis": {"status": "connected", "response_time_ms": 1.2}
  }
}
```

Why health checks matter:

- Docker uses them to know if containers are healthy
- Load balancers use them to route traffic
- Monitoring tools use them to alert when something breaks

3. `app/middleware.py` — Request Logging Middleware

Automatically logs EVERY API request without adding code to each endpoint:

```
2026-02-16 19:18:06 | INFO | api | GET /api/documents → 200 (45.23ms)
2026-02-16 19:18:07 | INFO | api | POST /api/query → 200 (234.56ms)
2026-02-16 19:18:08 | WARNING | api | GET /api/documents/xxx → 404 (2.10ms)
2026-02-16 19:18:09 | ERROR | api | POST /api/query → 500 (1502.33ms)
```

It automatically categorizes by status code:

- 200-399 → logger.info (everything fine)
- 400-499 → logger.warning (client error, like not found)
- 500+ → logger.error (server error, something broke)

4. app/main.py — Updated Main File

Added three things:

- Middleware registration (RequestLoggingMiddleware)
- Health routes (health.router)
- Startup/shutdown event logs

HOW TO ADD LOGGING TO ANY NEW FILE

Whenever you create a new file in the project, add logging in 2 steps:

python

```
# Step 1: Import at the top of the file
from core.logger import get_logger
logger = get_logger(__name__)

# Step 2: Use logger instead of print
logger.info("This is an info message")

logger.error(f"Something failed: {e}")
logger.warning("This might be a problem")

## WHERE LOGS ARE STORED
```

Terminal → Shows INFO, WARNING, ERROR in real-time
logs/ → Saves ALL levels (including DEBUG) to daily files
Example: logs/rag_2026-02-16.log

INTERVIEW TALKING POINTS

> "I implemented structured logging across the entire backend using Python's logging module. Every log entry includes a timestamp, severity level, module name, and message. I created a centralized logger that outputs to both the terminal and rotating log files. I also added request logging middleware that automatically tracks every API call with its method, path, status code, and response time. Health check endpoints monitor PostgreSQL and Redis connectivity with response time tracking, which Docker and monitoring tools use to verify system health."

Key terms: structured logging, log levels, middleware, health checks, observability, monitoring

QUICK REFERENCE

Log Levels:

logger.debug() → Debugging details (file only)
logger.info() → Normal operations
logger.warning() → Potential issues
logger.error() → Failures

Health Endpoints:

GET /health → Basic alive check
GET /health/detailed → Full dependency check

Log Format:

TIMESTAMP | LEVEL | MODULE | MESSAGE
2026-02-16 19:18:06 | INFO | api | GET /api/query → 200 (45ms)

Let's build it. I'll go step by step like before.

Step 1: Create the Logger

Path: backend/core/logger.py (new file inside core/ folder)

python

```
"""
Structured Logging System for Time-Aware RAG
Replaces all print() statements with proper logging.
Logs to both terminal and file with timestamps and levels.
"""

import logging
import sys
from pathlib import Path
from datetime import datetime

def setup_logger(name: str = "rag", log_to_file: bool = True) -> logging.Logger:
    """
    Create a structured logger.

    Args:
        name: Logger name
        log_to_file: Whether to also save logs to a file

    Returns:
        Configured logger instance
    """
    logger = logging.getLogger(name)

    # Avoid adding handlers multiple times
    if logger.handlers:
        return logger

    logger.setLevel(logging.DEBUG)

    # Log format with timestamp, level, and message
    formatter = logging.Formatter(
        fmt="%(asctime)s | %(levelname)-8s | %(name)s | %(message)s",
```

```

    datefmt="%Y-%m-%d %H:%M:%S"
)

# Handler 1: Print to terminal (INFO and above)
console_handler = logging.StreamHandler(sys.stdout)
console_handler.setLevel(logging.INFO)
console_handler.setFormatter(formatter)
logger.addHandler(console_handler)

# Handler 2: Save to file (DEBUG and above — captures everything)
if log_to_file:
    log_dir = Path("logs")
    log_dir.mkdir(exist_ok=True)

    log_filename = f"rag_{datetime.now().strftime('%Y-%m-%d')}.log"
    file_handler = logging.FileHandler(log_dir / log_filename)
    file_handler.setLevel(logging.DEBUG)
    file_handler.setFormatter(formatter)
    logger.addHandler(file_handler)

return logger

def get_logger(module_name: str = "rag") -> logging.Logger:
    """
    Get a logger for a specific module.

    Usage in any file:
    from core.logger import get_logger
    logger = get_logger(__name__)

    logger.info("Document added successfully")
    logger.error(f"Failed to connect: {e}")
    logger.warning("Cache miss")
    logger.debug("Processing query embedding")
    """
    return setup_logger(module_name)

```

Step 2: Create Health Check Endpoints

Path: backend/app/api/health.py (new file inside app/api/ folder)

python

```
"""
Health Check Endpoints
Used by Docker, monitoring tools, and load balancers
to check if the app and its dependencies are working.
"""

from fastapi import APIRouter
from core.database import engine
from core.cache import get_cache_manager
from core.logger import get_logger
import time

logger = get_logger("health")

router = APIRouter(tags=["Health"])

@router.get("/health")
async def basic_health():
    """
    Basic health check — is the app alive?
    Returns 200 if the server is running.
    """
    return {
        "status": "healthy",
        "service": "Time-Aware RAG API"
    }

@router.get("/health/detailed")
async def detailed_health():
```



```
"""
```

Detailed health check — checks all dependencies.
Shows status of PostgreSQL, Redis, and Vector Store.

```
"""
```

```
health = {  
    "status": "healthy",  
    "services": {}  
}
```

```
# Check PostgreSQL
```

```
try:
```

```
    start = time.time()  
    with engine.connect() as conn:  
        conn.execute("SELECT 1")  
    pg_time = (time.time() - start) * 1000
```

```
    health["services"]["postgresql"] = {  
        "status": "connected",  
        "response_time_ms": round(pg_time, 2)  
    }
```

```
    logger.info(f"PostgreSQL health OK ({pg_time:.2f}ms)")
```

```
except Exception as e:
```

```
    health["services"]["postgresql"] = {  
        "status": "disconnected",  
        "error": str(e)  
    }
```

```
    health["status"] = "degraded"
```

```
    logger.error(f"PostgreSQL health FAILED: {e}")
```

```
# Check Redis
```

```
try:
```

```
    cache = get_cache_manager()  
    start = time.time()
```

```
    if cache.enabled and cache.redis_client:
```

```
        cache.redis_client.ping()  
        redis_time = (time.time() - start) * 1000
```

```

        health["services"]["redis"] = {
            "status": "connected",
            "response_time_ms": round(redis_time, 2)
        }
        logger.info(f"Redis health OK ({redis_time:.2f}ms)")
    else:
        health["services"]["redis"] = {
            "status": "disabled"
        }
except Exception as e:
    health["services"]["redis"] = {
        "status": "disconnected",
        "error": str(e)
    }
    health["status"] = "degraded"
    logger.error(f"Redis health FAILED: {e}")

return health

```

Step 3: Create Request Logging Middleware

Path: backend/app/middleware.py (new file inside app/ folder)

python

```

"""
Request Logging Middleware
Automatically logs every API request with:
- Method (GET, POST, DELETE)
- Path (/api/query, /api/documents)
- Status code (200, 404, 500)
- Response time (how long it took)
"""

from starlette.middleware.base import BaseHTTPMiddleware
from starlette.requests import Request
from core.logger import get_logger

```

```
import time

logger = get_logger("api")

class RequestLoggingMiddleware(BaseHTTPMiddleware):
    async def dispatch(self, request: Request, call_next):
        # Record start time
        start_time = time.time()

        # Process the request
        try:
            response = await call_next(request)
        except Exception as e:
            # Log the error
            process_time = (time.time() - start_time) * 1000
            logger.error(
                f"{request.method} {request.url.path} → 500 ERROR ({process_time:.2f}ms) | {str(e)}"
            )
            raise

        # Calculate response time
        process_time = (time.time() - start_time) * 1000

        # Log based on status code
        status = response.status_code

        if status >= 500:
            logger.error(
                f"{request.method} {request.url.path} → {status} ({process_time:.2f}ms)"
            )
        elif status >= 400:
            logger.warning(
                f"{request.method} {request.url.path} → {status} ({process_time:.2f}ms)"
            )
        else:
```

```
        logger.info(
            f"{request.method} {request.url.path} → {status} ({{process_time:.2f}}ms)"
        )

    return response
```

Step 4: Update main.py to Use Logger, Middleware, and Health Routes

Path: backend/app/main.py — **replace entire file** with:

python

```
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware
from app.config import get_settings
from app.models import HealthCheck
from app.api import routes
from app.api import health
from app.middleware import RequestLoggingMiddleware
from core.logger import get_logger
import os

settings = get_settings()
logger = get_logger("main")

app = FastAPI(
    title="Time-Aware RAG API",
    description="Retrieval Augmented Generation with Temporal Intelligence",
    version="2.0.0"
)

# CORS
app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
```

```
    allow_credentials=True,
    allow_methods=["*"],

    allow_headers=["*"],
)

# Request logging middleware
app.add_middleware(RequestLoggingMiddleware)

# Include routes
app.include_router(routes.router)
app.include_router(health.router)

@app.on_event("startup")
async def startup_event():
    logger.info("🚀 Time-Aware RAG API starting up...")
    logger.info(f"📦 Embedding model: {settings.embedding_model}")
    logger.info(f"🗄️ Database: connected")
    logger.info(f"🗄️ Redis caching: {'enabled' if settings.enable_caching else 'disabled'}")
    logger.info("✅ API ready to serve requests")

@app.on_event("shutdown")
async def shutdown_event():
    logger.info("🛑 Time-Aware RAG API shutting down...")

if __name__ == "__main__":
    import uvicorn
    uvicorn.run(
        "app.main:app",
        host=settings.host,
        port=settings.port,
        reload=True
    )
```

Step 5: Update Dockerfile to Create Logs Directory

Path: backend/Dockerfile — find this line:

```
dockerfile
```

```
RUN mkdir -p data/faiss_index data/documents
```

Replace with:

```
dockerfile
```

```
RUN mkdir -p data/faiss_index data/documents logs
```

Step 6: Now Replace print() with logger in Core Files

This is the biggest step — we replace print() in your main files. I'll show the key files.

Path: backend/core/metadata_store.py — add at the **top of the file** (after the existing imports):

```
python
```

```
from core.logger import get_logger
logger = get_logger("metadata_store")
` ``
```

Then do a ****find and replace**** across the file:

```
` ``
```

```
print(f"✅ → logger.info(f"
```

```
print(f"❌ → logger.error(f"
```

```
print(f"⚠️ → logger.warning(f"
```

```
print(f"🔍 → logger.info(f"
```

```
print(f"🗑️ → logger.info(f"
```

Remove the emoji from the messages too. For example change:

```
python
```

```
print(f"✅ Added document to PostgreSQL: {doc_id[:8]}...")
```

to:

python

```
logger.info(f"Added document to PostgreSQL: {doc_id[:8]}...")
```

Do the same for these files — add the import at top and replace prints:

backend/core/cache.py — add at top:

python

```
from core.logger import get_logger  
logger = get_logger("cache")
```

backend/core/retriever.py — add at top:

python

```
from core.logger import get_logger  
logger = get_logger("retriever")
```

backend/core/embeddings.py — add at top:

python

```
from core.logger import get_logger  
logger = get_logger("embeddings")
```

backend/core/vector_store.py — add at top:

python

```
from core.logger import get_logger  
logger = get_logger("vector_store")
```

backend/llm/gemini_client.py — add at top:

python

```
from core.logger import get_logger  
logger = get_logger("gemini")
```

In each file, replace `print(` with the appropriate `logger.info(`, `logger.error(`, or `logger.warning(`.

Create these files, do the replacements, Then we'll rebuild Docker and test it!

done now how to test it

Rebuild and start Docker. Run from your backend folder:

bash

```
docker-compose up --build
```

If you still have the old containers running, first stop them:

bash

```
docker-compose down
docker-compose up --build
` ` `
```

Once it's running and you see "Uvicorn running on <http://0.0.0.0:8000>", open your browser and hit these:

1. **`http://localhost:8000/health`** — basic health check
2. **`http://localhost:8000/health/detailed`** — checks PostgreSQL and Redis
3. **`http://localhost:8000/docs`** — try the `/api/documents` endpoint

Then check your terminal — you should see structured logs like:

```
` ` `
2026-02-16 20:45:32 | INFO    | api | GET /health → 200 (1.23ms)
2026-02-16 20:45:33 | INFO    | api | GET /health/detailed → 200 (5.67ms)
```